

Actividad 1 · Del formulario a la base de datos · respuesta modelada (tutor)

Resumen de las actividades

	Actividad 1	Actividad 2
Nombre	Del formulario a la base de datos	Rescate del workflow roto
Técnica	Resolución de problemas	Análisis de caso + gamificación
AE que cubre	AE1, AE2, AE3	AE1, AE3, AE4
Indicadores	1.1, 2.1, 2.2, 2.3, 3.1, 3.2, 3.3, 3.4	1.1, 3.3, 4.1, 4.2, 4.3
Tramo del módulo	2 y 3 (semanas 1–2)	4 (semana 2–3)
Tiempo estimado	5 h (2 h guiadas + 3 h autónomas)	4 h (1,5 h sincrónica + 2,5 h autónomas)
Apoyos	Tutorial R05, cápsulas R03	Video interactivo R06
Producto	2 workflows exportados + captura de Supabase + nota breve	Workflow reparado + bitácora de depuración
Se evalúa con	Rúbrica de workflow (B2-1)	Rúbrica de workflow (B2-1) + bitácora (B4-d)

Parte A (ejemplo de respuesta de nivel logrado).

1. *Registrar cada pedido:* hoy se copia a mano; automatizarlo elimina la digitación y los pedidos perdidos o duplicados, y deja el registro disponible al instante.
2. *Validar y normalizar los datos:* correos con mayúsculas o comunas con espacios rompen los reportes; normalizar en la entrada evita corregir después.
3. *Consolidar las ventas por comuna para facturación:* hoy toma horas cada viernes y se equivoca; automatizado es un resumen exacto y en el formato que facturación exige.

Parte B — workflow de referencia "Pedido a registro" (4 nodos):

#	Nodo	Configuración clave
1	n8n Form Trigger	Título "Nuevo pedido". Campos con esas etiquetas exactas: <code>nombre</code> , <code>email</code> (tipo Email), <code>comuna</code> , <code>tipo_cliente</code> (Dropdown: minorista, mayorista), <code>producto</code> , <code>cantidad</code> (Number), <code>precio_unitario</code> (Number).
2	Edit Fields (Set)	Modo <i>Manual Mapping</i> . <code>email</code> (String) = <code>{{ \$json.email.trim().toLowerCase() }}</code> · <code>comuna</code> (String) = <code>{{ \$json.comuna.trim() }}</code> · <code>cantidad</code> (Number) = <code>{{ Number(\$json.cantidad) }}</code> · <code>precio_unitario</code> (Number) = <code>{{ Number(\$json.precio_unitario) }}</code> · <code>total</code> (Number) = <code>{{ Number(\$json.cantidad) * Number(\$json.precio_unitario) }}</code> · conservar <code>nombre</code> , <code>tipo_cliente</code> y <code>producto</code> .
3	Supabase	Credencial del proyecto del curso. Operación <i>Create a row</i> , tabla <code>pedidos</code> , <i>Auto-map input data to columns</i> .
4	Form Ending (opcional)	Mensaje "Pedido recibido. Te contactaremos a <code>{{ \$('Edit Fields').item.json.email }}</code> ".

Salida esperada del nodo 2 para un pedido de prueba (2 × Café en grano 1 kg a \$12.500, comuna escrita "Ñuñoa"):

```
{ "nombre": "Ana Rojas", "email": "ana.rojas@correo.test", "comuna": "Ñuñoa",  
  "tipo_cliente": "minorista", "producto": "Café en grano 1 kg",  
  "cantidad": 2, "precio_unitario": 12500, "total": 25000 }
```

Parte C — workflow de referencia "Resumen semanal" (6 nodos):

#	Nodo	Configuración clave
1	Manual Trigger	Para ejecutarlo a pedido (en producción se cambia por <i>Schedule Trigger</i> , viernes 18:00).
2	HTTP Request	GET al enlace del CSV. <i>Response Format</i> : File.
3	Extract from File	Operación <i>Extract From CSV</i> , primera fila como encabezados.
4	Filter	Condición: <code>{{ \$json.email }}</code> <i>is not empty</i> . Descarta la fila 4.
5	Edit Fields (Set)	<code>comuna = {{ \$json.comuna.trim() }}</code> · <code>total (Number) = {{ Number(\$json.cantidad) * Number(\$json.precio_unitario) }}</code> .
6	Summarize	<i>Fields to Split By</i> : comuna. Agregaciones: <i>Sum</i> de total y <i>Count</i> de id.
7	XML	Modo <i>JSON to XML</i> . Entrega el resumen como texto XML listo para facturación.

Resultado correcto del nodo 6 (7 pedidos válidos, \$566.540 en total):

comuna	pedidos	total
Ñuñoa	2	38.800
Maipú	2	208.600
Temuco	2	314.940
Valparaíso	1	4.200

Errores típicos y cómo se reconocen en la entrega:

- Sin el `trim()` del nodo 5 aparecen dos grupos: "Ñuñoa" con \$25.000 y "Ñuñoa " con \$13.800. Es la señal de que no se normalizó antes de agrupar (indicador 3.1).
- Sin el Filter aparecen 8 pedidos y Ñuñoa suma \$51.300 (indicador 3.3).
- Si `total` quedó como texto, el Summarize concatena en vez de sumar o marca error de tipo (indicador 3.3).
- Si Supabase rechaza la fila con `invalid input syntax for type integer`, cantidad o `precio_unitario` llegaron como texto (indicador 3.4).